

## Lezione 10 27/11/23 MVC/MCP

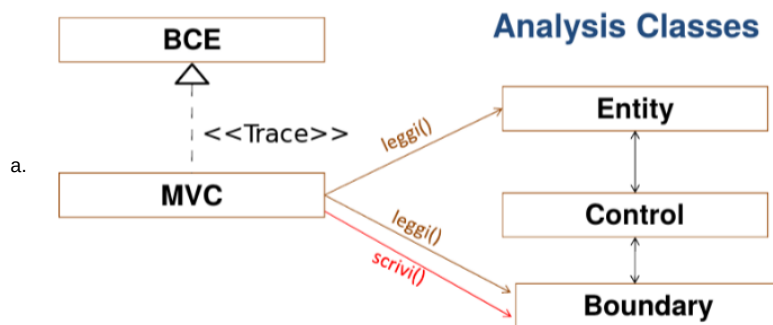
domenica 3 dicembre 2023 15:38

Vediamo ora come passare la modello dei casi d'uso alle classi di analisi , basandoci su una rappresentazione statica / vista del sistema : ricordiamoci il BCE :

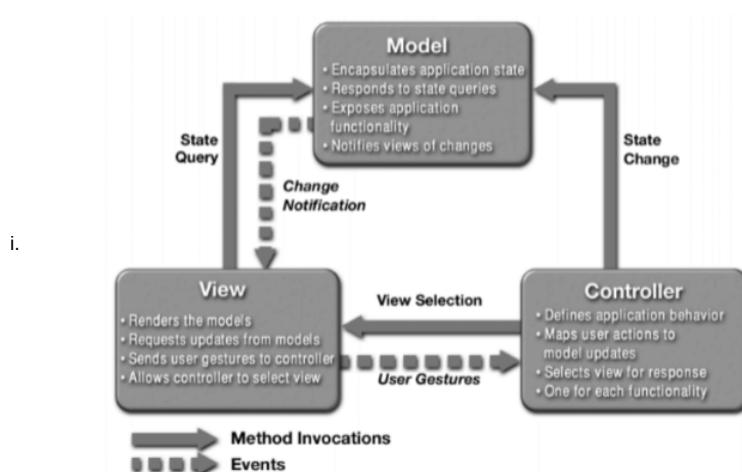


Dove le **boundary** mediano interazione attore sistema , rappresentando i confini del sistema , i **control** coordinano il comportamento durante il caso d'uso , rappresentando comportamenti dipendenti dal modo di interazione e sono trovati dall'analisi del dominio del problema ed infine troviamo le **entity** , ovvero astrazioni chiavi del sistema , dipendono dall'analisi del dominio del problema e modellano il comportamento di un'entità del dominio incapsulando i dati . Tutte e tre sono intese come macro classi . Notiamo che con questo pattern di analisi , **non si riesce a catturare tutte le problematiche** . Quindi in generale usiamo 2 varianti per catturare maggior problematiche : **MVC ( model view controller ) e MVP( model view presenter )** . In dettaglio :

1. MVC



- b. Quindi questo pattern traccia BCE ad un livello di astrazione più dettagliato : è possibile leggere informazioni da boundary ed entity. **Ci deve essere per forza le boundary** .
- c. In generale :

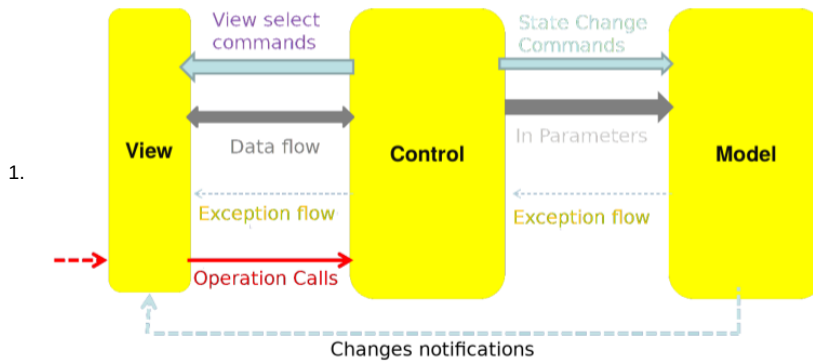


d. View :

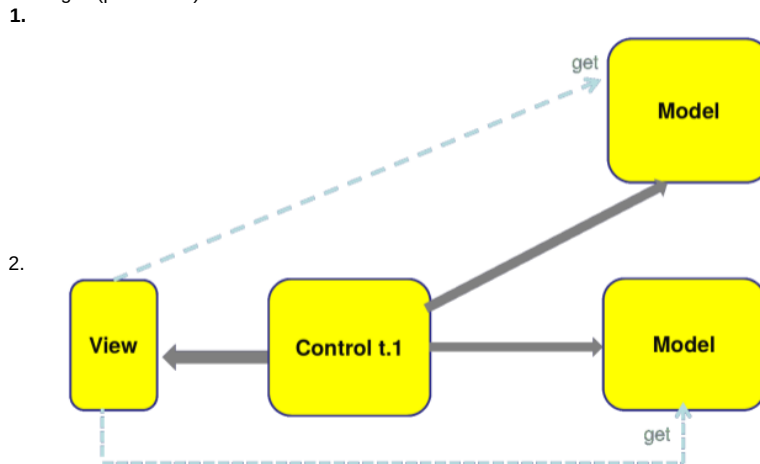
- i. Insieme degli artefatti software che servono per rappresentare la logica di rappresentazione dei dati : non confondiamole con le boundary (funzionalità) . Quindi le view hanno la responsabilità di costruzione / gestione interfaccia (anche quelle grafiche) . Accede in sola lettura alla parte del model bypassando boundary e control. Il suo compito principale è mantenere i "aggiornati" cioè presentarli anche se singolo cambiamento . Questo viene fatto attraverso due modelli : **push e pull** . Nel primo caso si ha che la view richiede dati aggiornati al model attraverso il pattern GOF OBSERVER : **quindi le view sono osservatori del model** . Nel secondo caso invece la view richiede i dati aggiornati al model solo quando necessario . Riassumendo : **le view sono classi partecipanti a questo "pattern" ma prevalentemente si intendono**

come quelle classi che si occupano solamente di rappresentare/acquisire i dati : linguaggio puramente dichiarativo , ovvero i famosi file .xml.

ii. In dettaglio (push-model) :



iii. In dettaglio (pull-model):



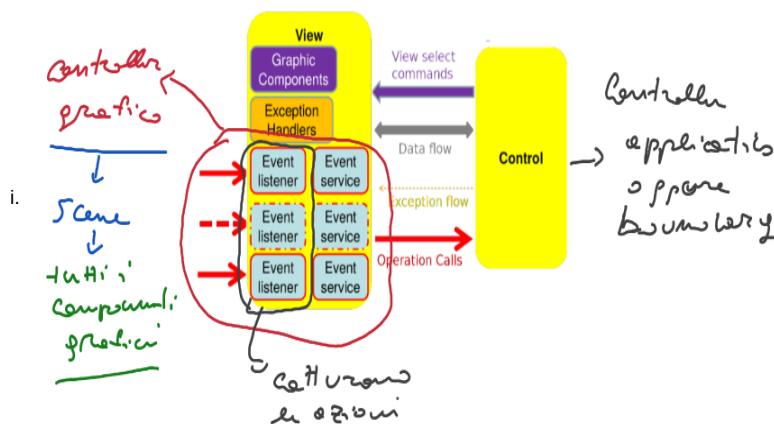
e. **Controller :**

- i. Queste classi vengono normalmente diviso in due "sotto classi" : **controller grafico e controller applicativo (grasp su view)**. Il primo ha la responsabilità di fare interagire la view (rappresentazione dei dati) con quello applicativo . Un esempio di questo passaggi può essere come fare la conversione tra dati esterni ed interni, ovvero scegliere come poter rappresentare una data . In generale questo controller fa parte della view. Quello **applicativo** matcha 1-1 con il controller classico di BCE : strato software che ha la responsabilità di implementare uno specifico caso d'uso . **Per quanto possibile deve essere stateless** . Fin qui tutto bello, ma in realtà dove lo metto ?? O li mettiamo in un unico bundle oppure li mettiamo in due bundle separati?? Se li mettiamo in un unico bundle si ha aumento dell'accoppiamento , quindi il controller (grafico + applicativo) tiene conto sia del come che del cosa . Quindi in generale se cambiamento della view , il controller cambia di conseguenza. Non viene usata praticamente, quindi andiamo a vedere il secondo modo : i controller grafici ed applicativi sono disaccoppiati : **il grafico invoca quello applicativo** , quindi possibile che 1 caso d'uso abbia più controlli grafici , ma uno solo applicativo. Con questa seconda modalità ed usiamo questa , è vero che aumenta lo sforzo , ma aumenta il disaccoppiamento . **Esempio se la view si aggiorna , si aggiorna solo il controller applicativo (lasciando inalterato quello grafico)** .

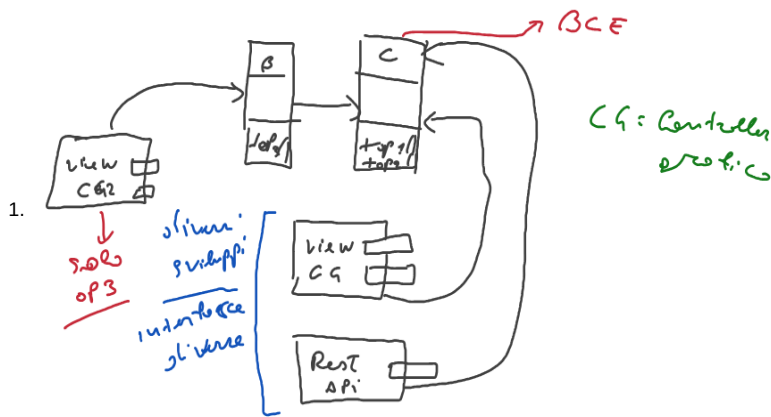
f. **Model :**

- i. Ricalca la parte di model di BCE: quindi costituisce la rete di entità partecipanti al caso d'uso : operazioni tra entità ed incapsulamento dei dati . Possibile estensione funzionalità del model usando il GOF OBSERVER .

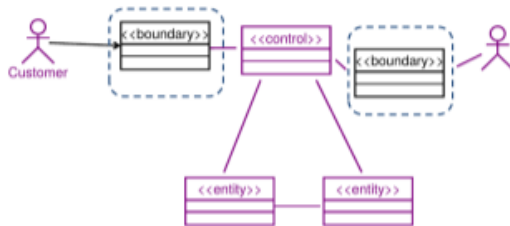
g. Quindi concettualmente lo schema è il seguente:



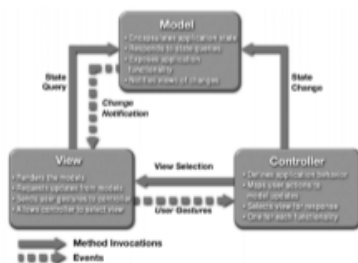
ii. Un esempio :



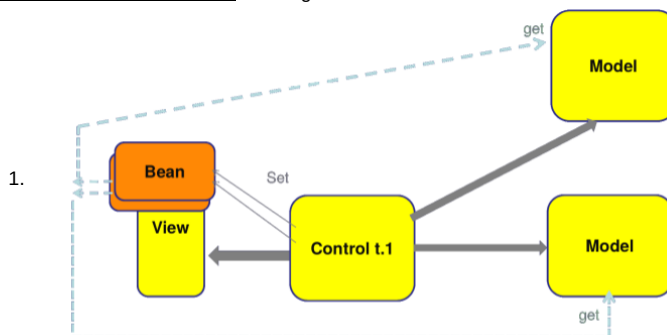
h. Riassumendo quindi :



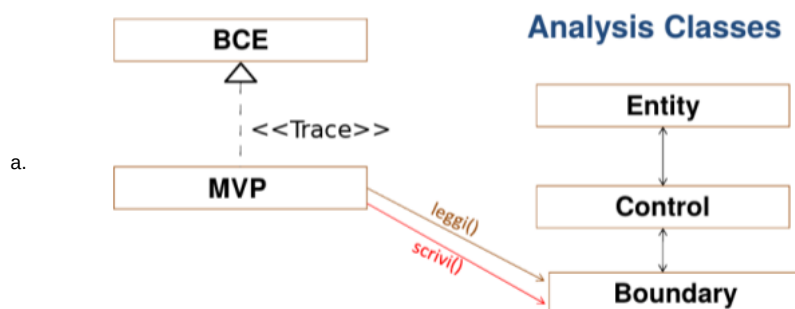
i.



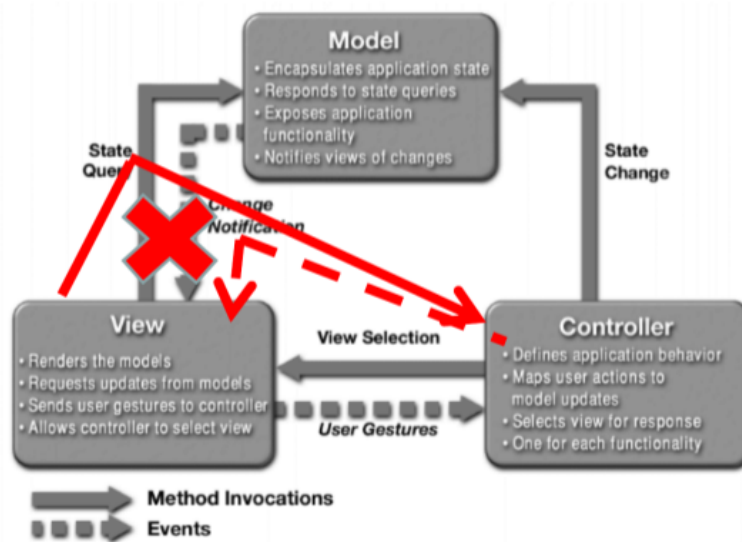
ii. Andiamo ora a vedere come e cosa succede se si hanno cambiamenti nel sistema : cambiamenti della rappresentazione delle informazioni , evoluzioni key abstraction con relative relazioni : si ha che si hanno cambiamenti view e/o boundary. Per ovviare a questi cambiamenti si usano i **bean** : tipo di dato fittizio che fa da buffer tra boundary control ed entity . Principalmente interposte tra le boundary ed il resto delle classi partecipanti al caso d'uso (VOPC). Con queste classi si ha disaccoppiamento entity e boundary... **fa da "controllore" sulla validazione sintattica dell'input** ed eventuale gestione dei riferimenti per i dati in visualizzazione. **Sono caratterizzati da attributi privati contenenti da i/o , metodi pubblici getters e setters , i quali permettono di accedere agli attributi ed hanno anche metodi privati per il controllo dei dati . Il cambiamento del model si ferma al bean.** In dettaglio :



2. In antitesi al mvc vi è il mvp :

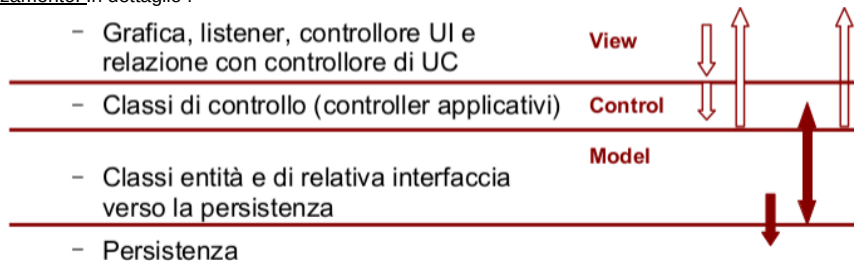


b.



- c. Quindi questo raffinamento di BCE è caratterizzato dalla non possibilità che i dati vengano acquisiti direttamente dal model, le view hanno accesso ai metodi del model attraverso i getter, l'interazione **obbligata** tra view e controller, mentre le restanti responsabilità sono pari a quelle di mvc. Per quanto riguarda il presentation model come controller di mvc, mentre il model è uguale a quello di mvc. Questo modello viene principalmente usato per sviluppare GUI.

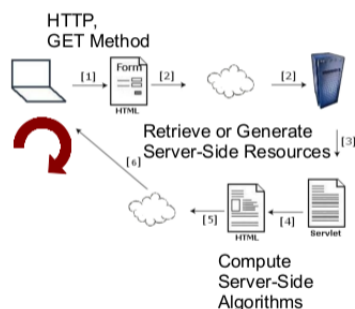
Ricordiamoci che tutte le "classi" sono sotto la stessa jvm, quindi stesso spazio di indirizzamento. In dettaglio:



- ⇔ Interazioni lettura/scrittura mediate da classi bean
- Interazioni lettura/scrittura

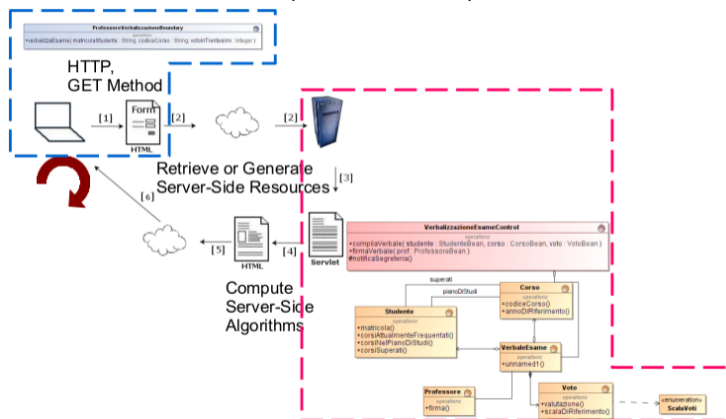
Nel dettaglio: abbiamo detto che il controller grafico invoca metodi dell'applicativo usando come parametri (attuali/formali) i bean. Invece il controller applicativo immagazzina i dati nel bean. Ricordiamoci che le bean sono delle camere stage, ovvero classi con attributi privati, metodi getter/setter e disaccoppiano dati interni da quelli esterni. Tutto questo vale per le stand-alone. Nelle distribuite invece come si lavora??? Partiamo dal concetto che l'applicazione coinvolge distinte unità computazionali: l'esecuzione delle applicazioni avviene su macchine differenti, quindi più jvm, il quale potrebbe portare ad un problema di tecnologia: potrebbe essere differente. Quindi per evitare questo mismatch, possiamo salvare una copia delle istanze con opportuni riferimenti, le quali verranno passate all'altra architettura. Quindi lo scambio di queste "copie di istanze" avviene attraverso uno scambio del binario del programma se soluzioni omogenee, oppure rappresentazioni portabili se formati macchine diverse. In dettaglio:

23

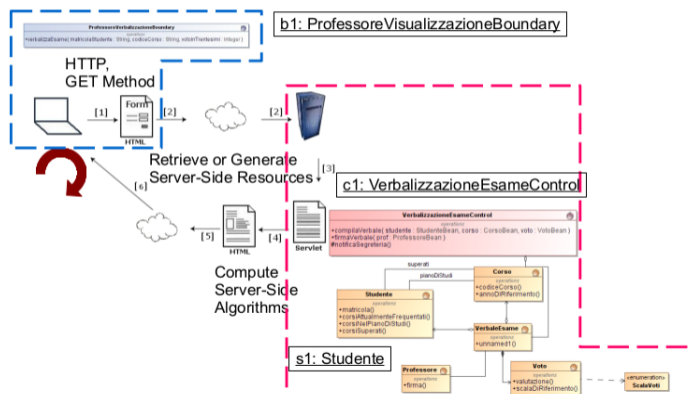


Notiamo che: le applicazioni distribuite forniscono funzionalità accessibili da remoto usando i **thin client**: web browser. La view (rappresentazione dei dati) non è istanziata sulle macchine utente, in quanto sono generalmente server side e trasferiti sul browser dei client per esecuzione (Applet java, html+jsp). Quindi non si possono trasferire istanze di entity e loro riferimenti (diversa jvm), ed i dati **devono per forza passare attraverso i bean**. Il controller ed il model (versione O.O) risiedono sulla macchina locale. Per quanto riguarda invece i controller

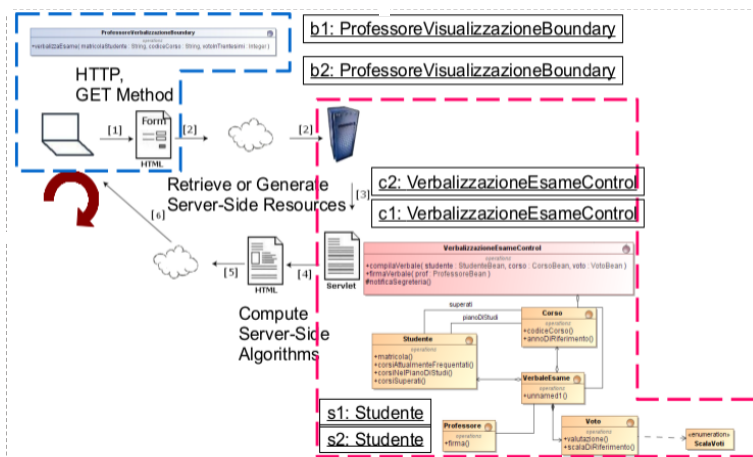
nelle architetture distribuite si parla di **SERVLET** : tirano su le istanze dei controller (più in generale di tutte le risorse che servono). Vediamo un esempio :



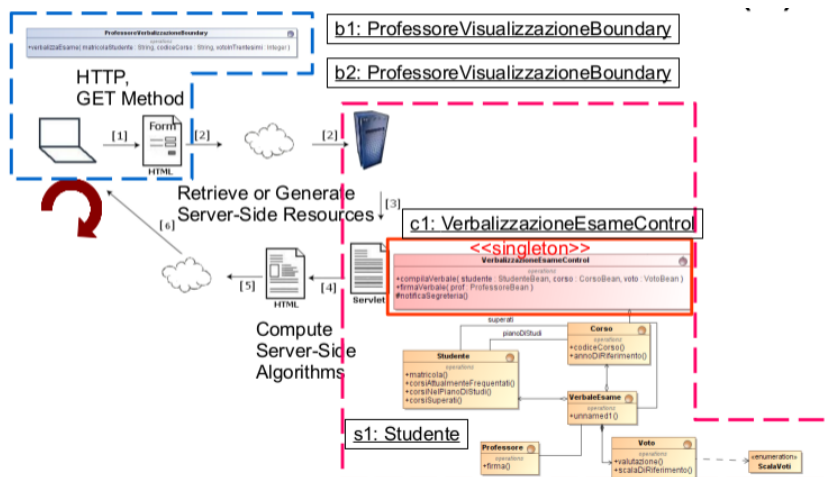
Simuliamo l'esecuzione :



Nel caso invece di più richieste ?? Più istanze : per ogni richiesta , si crea una nuova istanza :



► Vediamo che succede se a questo modello (corretto) se vi si applica il singleton (unica istanza nell'applicazione) ??



Quindi in questo preciso caso si ha un'anomalia : nonostante ad ogni richiesta si crea una nuova istanza degli oggetti richiesti , in questo caso si ha che vi è un'unica istanza del controller (dovuto al singleton) : **quindi ogni richiesta va a lavorare su quella stessa istanza**. Analogamente se il singleton viene applicato anche al model (studente) :

